TECHNICAL REPORT

The Mechanics of Multiple Linear Regression & Vectorisation

From Single-Feature Models to Vectorised, Multi-Feature Gradient Descent

1. Univariate → Multiple Regression 2. Vectors & Dot Products 3. The Magic of Vectorisation 4. Implementing with NumPy 5. Gradient Descent, Multi-Feature 6. Gradient Descent vs. Normal Equation 7. Summary & Practical Refinement

Prepared for: **Machine Learning Fundamentals** | Topic: **Multiple Linear Regression & Vectorisation**

01 From Univariate to Multiple Linear Regression

The transition from univariate to multiple linear regression marks the shift from using a single predictive variable to leveraging a suite of features to improve model accuracy. In a univariate model, the prediction is a simple linear function:

$$f = wx + b$$

Real-world phenomena — like the market value of a home — are rarely determined by a single factor. By expanding the model to include n features, we incorporate more information. For housing prices, we move beyond just "size" to include the number of bedrooms, the number of floors, and the age of the home. Each feature x_j is assigned its own weight w_j, giving the multiple linear regression formula:

$$f = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b$$

New Notation and Indexing

To manage multiple features, we adopt a specific mathematical notation. It is vital to distinguish between mathematical indexing (which typically starts at 1) and implementation indexing (which starts at 0 in Python/NumPy).

Notation	Definition	Math Index (j)	Code Index (j)
n	Total number of features	N/A	N/A
x_j	A specific feature	1 ... n	0 ... n−1
$\mathbf{x}^{(i)}$	Vector of all features for the i-th training example	i-th row	i
$x_j^{(i)}$	Value of feature j in training example i	1 ... n	0 ... n−1

Interpreting the Parameters: The Housing Example

Consider a model predicting housing prices in thousands of dollars:

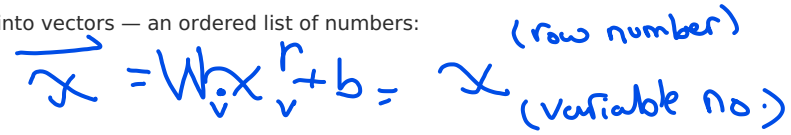
$$Price = 0.1x_1\ (size) + 4x_2\ (bedrooms) + 10x_3\ (floors) - 2x_4\ (age) + 80$$

- **b = 80** — the base price of a house is $80,000 (assuming zero size/age).
- **w_1 = 0.1** — for every additional square foot, the price increases by $100.
- **w_2 = 4** — each additional bedroom adds $4,000 to the value.
- **w_3 = 10** — each additional floor increases the price by $10,000.
- **w_4 = −2** — for every year of the house's age, the price decreases by $2,000.

02 The Mathematical Foundation: Vectors and Dot Products

To handle n parameters efficiently, we group them into vectors — an ordered list of numbers:

- Parameter Vector **w**: [..., w_j, ...]
- Feature Vector **x**: [..., x_j, ...]



In technical literature, you may see an arrow over the variable (e.g. $\vec{w}$ or $\vec{x}$) to signify it is a vector. It is a technical nuance worth noting that $\mathbf{x}^{(i)}$ is often represented as a row vector (a horizontal list) in data tables, though the mathematical operations remain the same.

The Dot Product

The relationship between weights and features is calculated using a dot product — the sum of the products of corresponding pairs:

$$\boldsymbol{w} \cdot \boldsymbol{x} = (w_1x_1) + (w_2x_2) + \ldots + (w_nx_n)$$

This lets us express the entire multiple linear regression model in a clean, compact form:

$$f_{\boldsymbol{w},b}(\boldsymbol{x}) = \boldsymbol{w} \cdot \boldsymbol{x} + b$$

03 The "Magic" of Vectorisation: Computational Efficiency

Vectorisation is the transition from sequential processing to parallel processing. Without vectorisation, a computer uses a for-loop, processing

index j = 0 at time T_0, then j = 1 at time T_1, and so on — highly inefficient for large models.



Behind the Scenes: Hardware and Libraries

The "magic" happens because modern hardware — CPUs and Graphics Processing Units (GPUs) — is designed for parallelism. GPUs were originally created to handle the massive parallel math required for computer graphics, and are now the backbone of machine learning. A vectorized operation uses numerical linear algebra libraries to:

1. **Parallel Multiply** — fetch all values of **w** and **x** and multiply them all at once in a single clock cycle.
2. **Specialized Summation** — use dedicated hardware to sum those results far faster than a sequential loop could.

This results in shorter code and massive speed gains. While the difference is small for 16 features, when scaling to thousands of features and millions of data points, vectorisation can reduce computation time from hours to minutes.

04 Implementing Vectorisation with NumPy

In Python, the **numpy** library is the industry standard for these operations.

For-Loop vs. Vectorised Code

PYTHON — NON-VECTORISED

```python
# Inefficient: sequential accumulation
f = 0
for j in range(n):
    f = f + w[j] * x[j]
f = f + b
```

PYTHON — VECTORISED

```python
# Efficient: single parallel operation
f = np.dot(w, x) + b
```

Derivative Updates

Vectorisation also streamlines the Gradient Descent update. Given 16 weights and 16 derivatives stored in vector **d**, all parameters can be updated in one line:

PYTHON

```python
w = w - 0.1 * d
```

Behind the scenes, the computer takes these arrays and uses parallel processing hardware to perform all 16 subtractions simultaneously.

05 Gradient Descent for Multiple Features

In multiple linear regression, Gradient Descent must minimize a cost function J(**w**, b) by updating every parameter w_j simultaneously. The update rule for a specific weight w_j is:

$$w_j = w_j - \alpha \, \partial/\partial w_j \; J(\mathbf{w}, b)$$

The derivative term is essentially an average of the error multiplied by the feature value. For a training set of m examples:

$$(1/m) \sum_{i=1}^{m} (f_{\mathbf{w},b}(\mathbf{x}^{(i)}) - y^{(i)}) \, x_j^{(i)}$$

This mirrors the univariate version, but the prediction $f_{\mathbf{w},b}(\mathbf{x}^{(i)})$ is now a dot product, and the error (f(x) − y) is multiplied by the specific feature x_j associated with that weight.

06 Comparative Analysis: Gradient Descent vs. The Normal Equation

While Gradient Descent is iterative, the Normal Equation provides a "one-shot" solution using advanced linear algebra.

Feature	Gradient Descent	Normal Equation
Method	Iterative (many small steps)	Non-iterative (solves all at once)
Generalization	Works for Linear, Logistic, and Neural Networks	Only works for Linear Regression
Scaling	Fast even if n is very large	Becomes very slow as n exceeds 10,000
Implementation	Standard for most ML practitioners	Often used "under the hood" by libraries

Expert Advice: Almost no machine learning practitioner should implement the Normal Equation themselves. While mature libraries may use it in the background for small-scale linear regression, Gradient Descent is the more versatile tool used across nearly all learning algorithms.

07 Final Summary and Practical Refinement

Vectorisation is not just a convenience; it is a foundational requirement for modern machine learning. By utilizing the parallel processing power of CPUs and GPUs through libraries like NumPy, we can scale our models to handle the complexity of the modern world.

Practical Tricks for Convergence

To make multiple linear regression truly effective, apply these practical techniques to help the algorithm converge:

- **Feature Scaling** — ensure all features (e.g. house size vs. number of bedrooms) occupy a similar range of values.
- **Learning Rate (α)** — carefully choose the step size so Gradient Descent finds the minimum efficiently without overshooting.

Takeaway: Multiple linear regression extends the single-feature model into a compact dot-product form, **f = w·x + b**, and vectorisation — via libraries like NumPy — turns slow, sequential for-loops into parallel hardware operations. Combined with feature scaling and a well-tuned learning rate, this makes Gradient Descent both fast and reliable, even as the number of features grows into the thousands.